

JAVA – Numeri e stringhe

Metodi Avanzati di Programmazione
Laurea Triennale in Informatica
Università degli Studi di Bari Aldo Moro
Docente: Pierpaolo Basile

Numbers

- JAVA mette a disposizione delle classi che rappresentano i tipi primitivi numerici
 - Byte, Short, Long, Integer, Float, Double
 - Ereditano tutte dalla classe Number
- Il compilatore converte automaticamente tra tipi primitivi e classi (boxing/unboxing)
- BigInteger/BigDecimal per calcolo ad alta precisione
- AtomicInteger/AtomicLong in applicazioni concorrenti

```
Integer ic = 3;  
int a = ic + 2;
```

Numbers

Tre motivi per utilizzare le classi Number

1. Quanto l'argomento di un metodo deve essere un oggetto e non un tipo primitivo
2. Utilizzare le costanti static definite nella classi (tipo MAX_VALUE, MIN_VALUE)
3. Per conversioni tra tipo
 - a) convertire String in numeri
 - b) conversioni tra differenti tipi numerici

Metodi implementati da Number

- Converta il valore in un tipo primitivo
 - `byte byteValue()`
 - `short shortValue()`
 - `int intValue()`
 - `long longValue()`
 - `float floatValue()`
 - `double doubleValue()`
- Confronta Number con l'argomento
 - `int compareTo(Byte anotherByte)`
 - `int compareTo(Double anotherDouble)`
 - `int compareTo(Float anotherFloat)`
 - `int compareTo(Integer anotherInteger)`
 - `int compareTo(Long anotherLong)`
 - `int compareTo(Short anotherShort)`
- `boolean equals(Object obj)`

>0 più grande
=0 uguali
<0 più piccolo

Metodi per la conversione

- `static Integer decode(String s)`: converte una stringa in `Integer`
- `static int parseInt(String s)`: converte una stringa in `int`
- `static int parseInt(String s, int radix)`: converte una stringa in `int`, (radix=sistema di numerazione 10, 2, 8, 16)
- `String toString()`: restituisce la stringa che rappresenta questo numero
- `static String toString(int i)`: restituisce la stringa che rappresenta il numero `i`
- `static Integer valueOf(int i)`: restituisce l'`Integer` che rappresenta il valore primitivo `i`
- `static Integer valueOf(String s)`: simile a `parseInt` ma restituisce un `Integer`
- `static Integer valueOf(String s, int radix)`: simile a `parseInt`

Stampa dei numeri

- Poiché ogni numero può essere convertito in String si possono utilizzare per la stampa dei numeri direttamente
 - `System.out.print(String s)`
 - `System.out.println(String s)`
- `System.out` è un oggetto `PrintStream`, possiamo usare i metodi *printf* e *format* (sono equivalenti)

format (printf)...

`format(String format, Object... args)`

Es.

```
System.out.format("The value of the float  
variable is %f, while the value of the integer  
variable is %d, and the string is %s",  
floatVar, intVar, stringVar);
```

- *format* specifica come gli oggetti *args* devono essere stampati

...format (printf)

Esempio

```
int i = 461012;  
System.out.format("The value of i is:  
%d%n", i);
```


format converter e flag

Converter	Flag	Explanation
d		A decimal integer
f		A float
n		A new line character appropriate to the platform running the application. You should always use %n, rather than \n
tB		A date & time conversion—locale-specific full name of month
td, te		A date & time conversion—2-digit day of month. td has leading zeroes as needed, te does not
ty, tY		A date & time conversion—ty = 2-digit year, tY = 4-digit year
tl		A date & time conversion—hour in 12-hour clock.
tM		A date & time conversion—minutes in 2 digits, with leading zeroes as necessary.
tp		A date & time conversion—locale-specific am/pm (lower case).
tm		A date & time conversion—months in 2 digits, with leading zeroes as necessary.
tD		A date & time conversion—date as %tm%td%ty
	08	Eight characters in width, with leading zeroes as necessary.
	+	Includes sign, whether positive or negative.
	,	Includes locale-specific grouping characters.
	-	Left-justified..
	.3	Three places after decimal point.
	10.3	Ten characters in width, right justified, with three places after decimal point.

format (esempio)

```
import java.util.Locale;

public class TestFormat {
    public static void main(String[] args) {
        long n = 461012;
        System.out.format("%d%n", n);          // --> "461012"
        System.out.format("%08d%n", n);        // --> "00461012"
        System.out.format("%+8d%n", n);        // --> " +461012"
        System.out.format("% ,8d%n", n);       // --> " 461,012"
        System.out.format("%+,8d%n%n", n);     // --> "+461,012"
        double pi = Math.PI;
        System.out.format("%f%n", pi);          // --> "3.141593"
        System.out.format("%.3f%n", pi);        // --> "3.142"
        System.out.format("%10.3f%n", pi);      // --> "      3.142"
        System.out.format("%-10.3f%n", pi);     // --> "3.142"
        System.out.format(Locale.FRANCE, "%-10.4f%n%n", pi); // --> "3,1416"
    }
}
```

Math

- La classe *Math* fornisce strumenti più avanzati per le operazioni matematiche
- Costanti matematiche
 - `Math.E`, `Math.PI`
- Operazioni di base
 - `abs` (valore assoluto), `round` (arrotondamento), `min`, `max` (tra due numeri)
 - *`double ceil(double d)`*: l'intero più piccolo che è più grande o uguale a *d*
 - *`double floor(double d)`*: l'intero più grande che è più piccolo o uguale a *d*
 - *`double rint(double d)`*: l'intero più vicino a *d*

Math (esponenziali e logaritmi)

- `double exp(double d):` e^d
- `double log(double d):` $\ln(d)$
- `double pow(double base, double exponent):`
 $\text{base}^{\text{exponent}}$
- `double sqrt(double d):` radice quadrata di d

Math (funzioni trigonometriche)

Passare come argomento un double che rappresenta l'angolo espresso in radianti

- *sin, cos, tan*
- *asin, acos, atan* (arcoseno, ...)
- *double atan2(double y, double x)*: converte coordinate rettangolari (x, y) in coordinate polari (r, theta) e restituisce theta
- *double toDegrees(double d), double toRadians(double d)*: converte l'argomento in gradi o radianti

Numeri random

- `double Math.random()`: restituisce un numero random tra `[0; 1)`
 - si può moltiplicare il risultato per un intero per scalare il range *`Math.random()*10`*
- Per creare una serie di numeri random utilizzare la classe *`java.util.Random`*

STRING

La classe Character

- In genere i caratteri vanno gestiti con il tipo primitivo char (char a = 'a')
- Metodi statici della classe Character

boolean isLetter(char ch) boolean isDigit(char ch)	determina se il carattere è una lettera o una cifra
boolean isWhitespace(char ch)	determina se il carattere è un white space (spazio, accapo, tabulazione)
boolean isUpperCase(char ch) boolean isLowerCase(char ch)	determina se il caratter è in maiuscolo (UpperCase) o minuscolo (LowerCase)
char toUpperCase(char ch) char toLowerCase(char ch)	determina la maiuscola e la minuscola del corrispettivo carattere
toString(char ch)	restituisce una stringa che rappresenta un singolo carattere

String

- La classe `String` rappresenta una stringa di caratteri
- Un'istanza di `String` è **immutabile**: il suo valore non può essere modificato dopo la creazione
 - utilizzare `StringBuilder` o `StringBuffer`
- Ogni literal “sdjfl skf dskj” è rappresentato in JAVA da un'istanza di `String`

String

String può essere visto come un array di char

```
char[] helloArray = { 'h', 'e', 'l', 'l', 'o', '.' };  
String helloString = new String(helloArray);  
System.out.println(helloString);
```

Il metodo `charAt(int i)` di `String` restituisce il carattere alla *i*-esima posizione

String (metodi)

- `length()`
 - restituisce la lunghezza della stringa in caratteri
- `contains(CharSequence s)`
 - restituisce true se contiene la sottostringa s
- `indexOf(String s)`
 - restituisce l'indice dal quale inizia la sottostringa s, -1 se non esiste la sottostringa
- `indexOf(String s, int i)`
 - inizia la ricerca della sottostringa a partire dall'*i*-esimo carattere

String (metodi)

- `replace(CharSequence s1, CharSequence s2)`
 - sostituisce la sequenza `s1` con `s2`
- `replaceAll(String regex, String r)`
 - sostituisce tutte le sequenze che corrispondono all'espressione regolare `regex` con `r`
- `matches(String regex)`
 - restituisce `true` se la stringa corrisponde all'espressione regolare `regex`
- `split(String regex)`
 - restituisce un array di `String` dividendo dove c'è il match con `regex`

String (metodi)

- `startsWith, endsWith`
 - controlla se inizia/finisce con una particolare sequenza
- `equals(Object o)`
 - operatore di uguaglianza, da usare per confrontare due stringhe **NON USARE ==**
- `int compareTo(String str), int compareToIgnoreCase(String str)`
 - confronta due stringa in modo

String (metodi)

- `substring(int b, int e)`,
`substring(int b)`
 - genera una sottostringa da *b* ad *e* (escluso),
genera una sottostringa a partire da *b* (incluso)
- `trim()`
 - elimina white space dall'inizio e dalla fine
- `toLowerCase()`, `toUpperCase()`
 - conversione minuscolo/maiuscolo

String (esempio)

```
public class StringDemo {  
    public static void main(String[] args) {  
        String palindrome = "Dot saw I was Tod";  
        int len = palindrome.length();  
        char[] tempCharArray = new char[len];  
        char[] charArray = new char[len];  
        // put original string in an  
        // array of chars  
        for (int i = 0; i < len; i++) {  
            tempCharArray[i] =  
                palindrome.charAt(i);  
        }  
        // reverse array of chars  
        for (int j = 0; j < len; j++) {  
            charArray[j] =  
                tempCharArray[len - 1 - j];  
        }  
        String reversePalindrome =  
            new String(charArray);  
        System.out.println(reversePalindrome);  
    }  
}
```

Inversione dei caratteri in
una stringa

Conversione da stringhe a numeri

```
int i = Integer.parseInt("42");  
float f = Float.parseFloat("3.14");  
double d = Double.parseDouble("4.32144");
```

```
Float fo = Float.valueOf("3.14");
```


Conversione da numeri a stringhe

```
int i=3;  
double d=3.4;  
String s3 = Integer.toString(i);  
String s4 = Double.toString(d);
```

```
int i=3;  
String s1 = String.valueOf(i);
```

Esercizio

1. Prendere due stringhe passate come argomento al main
2. Convertirle in double
3. Effettuare la somma dei due double e stamparla a video
4. Effettuare l'elevamento a potenza di $\text{arg1}^{\text{arg2}}$ e stamparlo a video
5. Se il primo numero è minore di 100 genera un numero random tra 0 e 100 e stampare true se il primo numero è più piccolo del numero random, altrimenti stampare false

Espressioni regolari

- Un'Espressione regolare è una parola che denota un linguaggio regolare
 - può essere utilizzate per verificare se una parola (String) corrisponde alle regole di un linguaggio
 - Es. a^+ denota tutte le stringhe composte da sole a
- Nella javadoc della classe Pattern trovate una sintesi sulle espressioni regolari
- JAVA tutorial
<http://docs.oracle.com/javase/tutorial/essential/regex/>

Espressioni regolari (caratteri)

<i>x</i>	The character <i>x</i>
<i>\\</i>	The backslash character
<i>\0n</i>	The character with octal value <i>0n</i> ($0 \leq n \leq 7$)
<i>\0nn</i>	The character with octal value <i>0nn</i> ($0 \leq n \leq 7$)
<i>\0mnn</i>	The character with octal value <i>0mnn</i> ($0 \leq m \leq 3, 0 \leq n \leq 7$)
<i>\xhh</i>	The character with hexadecimal value <i>0xhh</i>
<i>\uhhhh</i>	The character with hexadecimal value <i>0xhhhh</i>
<i>\t</i>	The tab character (' <i>\u0009</i> ')
<i>\n</i>	The newline (line feed) character (' <i>\u000A</i> ')
<i>\r</i>	The carriage-return character (' <i>\u000D</i> ')
<i>\f</i>	The form-feed character (' <i>\u000C</i> ')
<i>\a</i>	The alert (bell) character (' <i>\u0007</i> ')
<i>\e</i>	The escape character (' <i>\u001B</i> ')
<i>\cx</i>	The control character corresponding to <i>x</i>

Espressioni regolari (chars classes)

[abc] a, b, or c (simple class)

[^abc] Any character except a, b, or c (negation)

[a-zA-Z] a through z or A through Z, inclusive
(range)

[a-d[m-p]] a through d, or m through p: [a-dm-p]
(union)

[a-z&&[def]] d, e, or f (intersection)

[a-z&&[^bc]] a through z, except for b and c: [a-d-z]
(subtraction)

[a-z&&[^m-p]] a through z, and
not m through p: [a-lq-z](subtraction)

Espressioni regolari (default classes)

- Any character (may or may not match line terminators)

\d A digit: [0-9]

\D A non-digit: [^0-9]

\s A whitespace character: [\t\n\x0B\f\r]

\S A non-whitespace character: [^\s]

\w A word character: [a-zA-Z_0-9]

\W A non-word character: [^\w]

Espressioni regolari (boundary)

- ^ The beginning of a line
- \$ The end of a line
- \b A word boundary
- \B A non-word boundary
- \A The beginning of the input
- \G The end of the previous match
- \Z The end of the input but for the final terminator, if any
- \z The end of the input

Espressioni regolari (greedy operator)

$X?$ X , once or not at all

X^* X , zero or more times

X^+ X , one or more times

$X\{n\}$ X , exactly n times

$X\{n,\}$ X , at least n times

$X\{n,m\}$ X , at least n but not more than m times

Espressioni regolari (quotation)

- \ Nothing, but quotes the following character
- \Q Nothing, but quotes all characters until \E
- \E Nothing, but ends quoting started by \Q

Espressioni regolari (logical operators)

XY X followed by Y

$X|Y$ Either X or Y

(X) X , as a capturing group

Esercizio

1. Definire l'espressione regolare per riconoscere un indirizzo email
2. Prendere il primo argomento passato al main
3. Se l'argomento è un indirizzo email stampa true altrimenti stampa false

StringBuilder

- E' una classe simile a String ma a lunghezza variabile
 - posso sempre aggiungere nuovi caratteri
- Metodi
 - length(): restituisce la lunghezza della stringa presente nel StringBuilder
 - capacity(): restituisce la capacità attuale di questo StringBuilder ($\geq \text{length}()$)

StringBuilder (costruttori)

Costruttore	Descrizione
<code>StringBuilder()</code>	Crea uno <code>StringBuilder</code> vuoto
<code>StringBuilder(CharSequence cs)</code>	Crea uno <code>StringBuilder</code> a partire da una sequenza di caratteri
<code>StringBuilder(int initCapacity)</code>	Crea uno <code>StringBuilder</code> con una capacità iniziale <i>initCapacity</i>
<code>StringBuilder(String s)</code>	Crea uno <code>StringBuilder</code> a partire da una stringa <i>s</i>

StringBuilder (metodi)...

- *StringBuilder append(...)*: aggiunge alla fine dello `StringBuilder` l'argomento (viene convertito in `String` anche i tipi primitivi)
- *StringBuilder delete(int start, int end)*,
StringBuilder deleteCharAt(int index): cancella una porzione o un carattere
- *StringBuilder insert(int offset, ...)*: inserisce l'argomento a partire dalla posizione *offset*

...StringBuilder (metodi)

- *StringBuilder replace(int start, int end, String s), void setCharAt(int index, char c)*: sostituisce una porzione o un singolo carattere
- *StringBuilder reverse()*: inverte l'ordine dei caratteri
- *String toString()*: restituisce una stringa che contiene la sequenza dei caratteri in `StringBuilder`

StringBuilder (esempio)

```
public class StringBuilderDemo {  
    public static void main(String[] args) {  
        String palindrome = "Dot saw I was Tod";  
        StringBuilder sb = new StringBuilder(palindrome);  
        sb.reverse(); // reverse it  
        System.out.println(sb);  
    }  
}
```

Inversione di una stringa
utilizzando StringBuilder

Approfondimento

ESPRESSIONI REGOLARI

Il metodo `split` di `String`

- Il metodo `split` della classe `String` suddivide una stringa in tante sottostringhe in base all'espressione regolare passata come parametro
- Per esempio se si vuole suddividere una stringa in base agli spazi:

```
String test1 = "sdfkljsd lkjsd fkl    lkdsjf  fs1kdjf f  sldkj";  
String[] split = test1.split("\\s+");  
for (String s : split) {  
    System.out.println(s);  
}
```

Il metodo `replaceAll` di `String`

- Il metodo `replaceAll` della classe `String` sostituisce tutte le sottostringhe che corrispondono ad una espressione regolare con una stringa passata come parametro
- Per esempio se si vogliono sostituire sequenze di numeri con uno spazio:

```
String test2 = "sdfkljsd824lkjsd76fkl2765lksdjf549fs1kdjf";  
String tt2 = test1.replaceAll("[0-9]+", " ");
```

Classe Pattern

- La classe Pattern è una rappresentazione compilata di un'espressione regolare
- Un'espressione regolare, specificata come stringa, deve prima essere compilata in un'istanza di questa classe
- Il modello risultante può quindi essere utilizzato per creare un oggetto Matcher capace di fare il matching tra qualsiasi sequenza di caratteri e l'espressione regolare compilata
- Un esempio tipico di invocazione è:

```
Pattern p = Pattern.compile("a*b");  
Matcher m = p.matcher("aaaaab");  
boolean b = m.matches();
```

Classe Matcher

- Esegue le operazioni di matching su una sequenza di caratteri in funzione dell'espressione regolare compilata
- Un matcher viene creato da un pattern invocando il metodo `matcher` del pattern. Una volta creato, un matcher può essere utilizzato per eseguire tre diversi tipi di operazioni di match:
 - Il metodo **`match`** tenta di abbinare l'intera sequenza di input al pattern
 - Il metodo **`lookingAt`** cerca la sequenza di input corrispondente all'espressione regolare, a partire dall'inizio della stringa
 - Il metodo **`find`** analizza la sequenza di input cercando la sottosequenza successiva che corrisponde all'espressione regolare
- Ognuno di questi metodi restituisce un valore booleano che indica l'esito positivo o negativo. Maggiori informazioni possono essere ottenute interrogando lo stato del matcher

Classe Matcher (stato)

- Un matcher trova corrispondenze in un sottoinsieme del suo input chiamato regione. Per impostazione predefinita, la regione contiene tutto l'input del matcher
 - La regione può essere modificata tramite il metodo `region` e interrogata tramite i metodi `regionStart` e `regionEnd`
- Lo stato esplicito di un matcher include gli indici di inizio e fine dell'ultimo match (`start` ed `end`)
- Include anche gli indici di inizio e fine della sottosequenza dell'ultimo match, nonché un conteggio totale (`groupCount`) di tali sottosequenze
 - Per comodità, vengono forniti anche metodi per restituire queste sottosequenze in forma di stringa (`group`)

Utilizzo di Matcher

```
Matcher matcher1 = pattern.matcher("dlkf1sASDas1sdSD");  
//deve corrispondere l'intera stringa  
System.out.println(matcher1.matches());
```

```
Matcher matcher2 = pattern.matcher("dlkf1sASDas1sdSD 8798767");  
//la corrispondenza deve partire dall'inizio ma non è  
necessario che corrisponda l'intera stringa  
System.out.println(matcher2.looksAt());
```

Utilizzo di Matcher

```
Matcher matcher3 = pattern.matcher("dlkf1sASDas1sdSD");  
//cicla su tutti i matching  
while (matcher3.find()) {  
    System.out.println(matcher3.group() + ": " +  
        matcher3.start() + "-" + matcher3.end());  
}
```


Utilizzo di Matcher

```
//una sequenza di numeri seguita da 1 o massimo 3 lettere min.  
String regexp = "([0-9]+)([a-z]{1,3})";  
Pattern pattern2 = Pattern.compile(regexp);  
String str = "9843989jf 39203920jie 32122i";  
Matcher matcher4 = pattern2.matcher(str);  
while (matcher4.find()) {  
    //restituisce il numero di gruppi (individuati dalle  
    //parentesi nella regexp)  
    int gc = matcher4.groupCount();  
    //il gruppo 0 corrisponde all'intero matching  
    for (int i = 0; i <= gc; i++) {  
        System.out.println(matcher4.group(i) + ": " +  
            matcher4.start(i) + "-" + matcher4.end(i));  
    }  
    System.out.println();  
}
```

THE END

